

This is a copy of a shared ChatGPT chat. It won't be added to memory.
Report conversation

Today 5:33 PM

i want to make a logo based on this design.



Edit

Branched from [Branch · Logo Design Request](#)



Edit

this logo is intended to be for a product of linear project management. I'd like to develop this idea further and so initially want to take these ideas and put them together (and expand on them) into a short summary of the method, with diagrams where helpful. the idea is that most activities are cycles similar to PDSA but can be "unrolled" onto a linear timeline. focusing on some part of the timeline, as a sliding window, is just a natural part of the project progression. tasks are kept in a strictly linear order, with only some basic hierarchical tools that should only be used for further decomposing a task

Show more ▾



Linear Project Management

Linear Project Management starts from a simple observation:

Work happens over time. Why not organize it that way?

Most project-management systems organize work primarily by **state**: backlog, ready, doing, blocked, done, someday, version 2, post-launch, and so on.

Linear Project Management instead makes **sequence** the primary organizing principle.

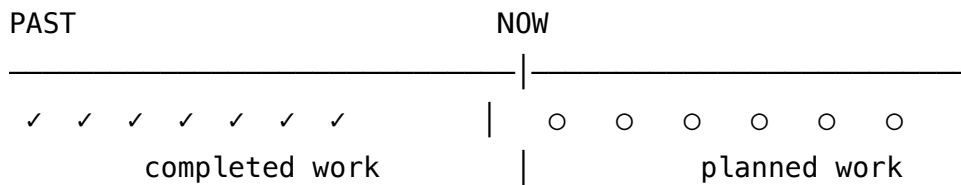
Every meaningful piece of work occupies a position in one ordered stream:

past → present → future

The project advances through that stream continuously.

1. The Project Is a Line

At its simplest, a project can be represented as:



There is no permanent "Done" column accumulating hundreds of cards.

Done work simply becomes **history**.

Likewise, future work doesn't need to be divided prematurely into an elaborate collection of buckets. It exists ahead of the present, in the order in which we currently believe it should happen.

The line itself becomes the durable record of the project.

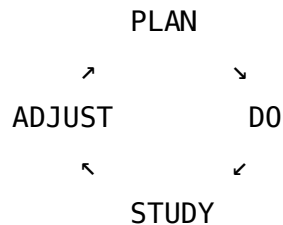
2. Work Is Cyclical, but Progress Is Linear

Much human activity is cyclical.

We:

Plan → Do → Study → Adjust → Plan → Do → Study → Adjust →
...

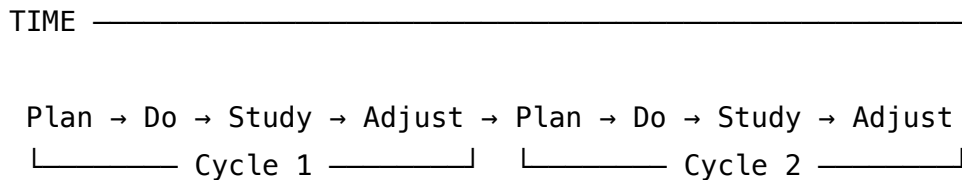
This resembles Deming's PDSA cycle and many other iterative models.



But a project does not literally travel in circles.

Time moves forward.

So the same cycle can be **unrolled**:



This distinction is important.

The method is iterative without requiring the project structure itself to be cyclical.

Every experiment, decision, adjustment, release, mistake, discovery, and subsequent response becomes another point on the project's linear history.

The loop is how we **learn**.

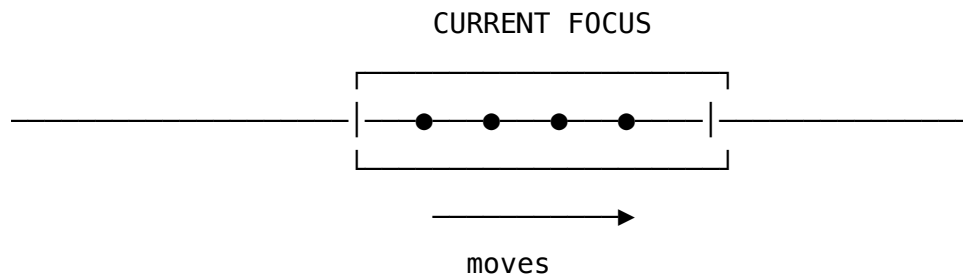
The line is how we **move**.

3. The Sliding Window

Nobody needs to think about the entire history and future of a

project simultaneously.

Instead, attention operates as a **sliding window** over the line.



At the beginning of a project, the window naturally contains early exploration and foundational work.

Later it contains implementation.

Later still it may contain deployment, observation, refinement, support, or another round of discovery.

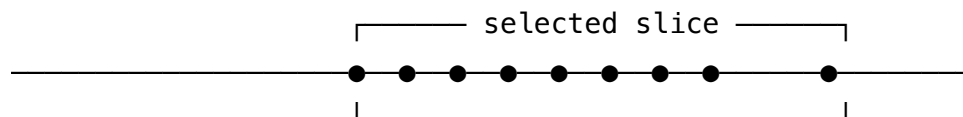
The underlying project hasn't been reorganized.

Our attention simply moved.

This provides a natural answer to something other systems often represent through board manipulation, milestones, iterations, sprints, archived cards, or proliferating columns.

4. Any Slice Tells a Story

Because work remains ordered, any section of the line can be inspected independently.



Look backward and the slice answers:

What happened during this period?

Look forward and it answers:

What do we currently expect to happen during this period?

Look around the present and it answers:

Where are we, how did we get here, and what comes next?

This makes the timeline simultaneously:

- the plan,
- the working queue,
- and the historical record.

The distinction between them is primarily **where you look**.

5. Strict Ordering

Tasks have an explicit linear order.

A → B → C → D → E → F → G

That does **not** necessarily mean every task has a hard dependency on the one immediately before it.

It means the team has made an explicit statement:

Given what we know now, this is the order in which we intend to address the work.

That order can change as we learn.

Changing it is not a failure of planning. It **is planning**.

The important property is that at any moment the project expresses a coherent ordering rather than an assortment of loosely prioritized buckets.

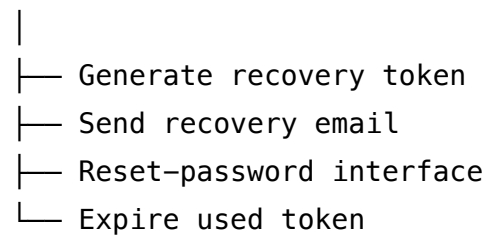
6. Hierarchy Exists—but It Should Be Rare

Sometimes one piece of work genuinely consists of several

pieces that belong together.

In those cases, hierarchy is useful:

Build account recovery



But hierarchy should mean something stronger than:

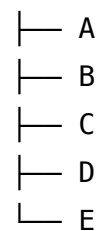
"These things seem related."

The parent represents **one unit of work whose constituent parts must collectively be completed.**

If the children can meaningfully be prioritized, delivered, deferred, or evaluated independently, they probably belong directly on the line.

Instead of:

Feature X



Prefer:



This is more common than traditional project structures tend to acknowledge.

Deep task trees can create an attractive fiction that the shape of the work is known in advance.

The line encourages decomposition **only when decomposition helps us do the work.**

7. Avoid Temporal Buckets

Traditional boards have a tendency to accumulate places that effectively mean:

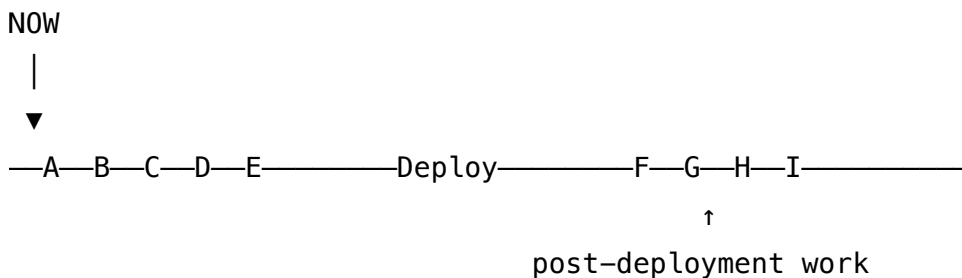
"not now."

Backlog | Ready | Doing | Blocked | Done | Post-Go-Live |

"Blocked," "post-go-live," "version 2," "later," and similar categories can gradually become disguised **tomorrow buckets.**

Linear ordering provides another answer.

If something should happen later, **put it later.**



"Post-go-live" doesn't necessarily need to be a container.

It can simply describe **where the work sits on the line.**

Likewise, a blocked item does not automatically require a permanent geographic region of the workflow. Its position, status, and relationship to surrounding work can communicate what matters without creating another destination where work can disappear.

8. History Is a First-Class Property

A Kanban board is excellent at answering:

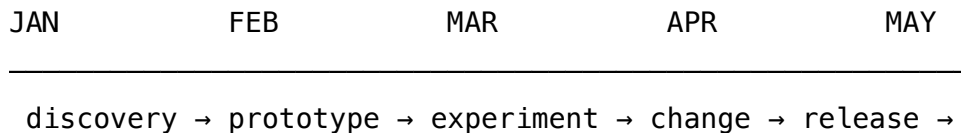
What state is work in right now?

It is less naturally suited to answering:

What did the project actually look like six months ago?

Eventually the Done column becomes enormous and completed work gets archived or otherwise removed from the primary view.

A linear system doesn't have this problem.



Old work isn't clutter.

It's farther away.

Zoom out to understand the project.

Zoom in to understand a period.

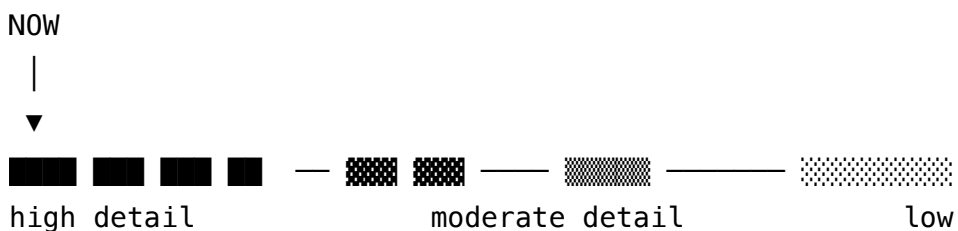
Move backward to investigate history.

Move forward to inspect the plan.

Return to **Now** to work.

9. Planning Is Progressive

The future should become less detailed as distance from the present increases.



Near-term work can be concrete because we know more about it.

Far-future work should often remain coarse because pretending otherwise creates false certainty.

As the window moves forward, distant ideas approach the present and are progressively explored, decomposed, reordered, clarified—or discarded.

This makes **rolling-wave planning** an inherent property of the representation rather than a separate planning ceremony.

10. The Core Model

The system can therefore remain surprisingly small.

Conceptually, we need:

1. **A line** — the ordered history and future of the project.
2. **Work items** — things worth doing.
3. **A Now position** — the boundary between history and intention.
4. **A sliding window** — the portion currently receiving attention.
5. **Limited hierarchy** — decomposition where the pieces genuinely form one whole.
6. **Lightweight state/metadata** — enough to describe exceptional conditions without turning those conditions into places where work lives.

Everything else should have to justify its existence.

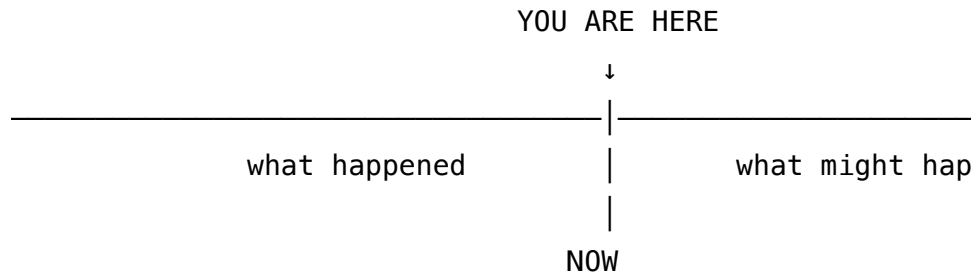
A Different Mental Model

Kanban asks us to imagine work moving across a board:



Linear Project Management asks us to imagine **ourselves moving**

through the work:



That inversion may be the central idea.

Work doesn't have to move toward Done.

Time moves us through the work.

The result is a project-management model built around sequence, history, learning, and the simple reality that every project has exactly one direction:

forward.

what can we call it? is the term "Linear project management" an existing idea? if so, are we talking about something similar? if not, can we come up with a better name for it?

I think **"Linear Project Management"** is **probably the wrong name**, even though *linear* is exactly the important visual metaphor in your idea.

The problem is that the term already has baggage. In project-management literature, "linear" is commonly used as essentially a synonym for **predictive / waterfall**: sequential phases, substantial up-front planning, fixed scope, and limited iteration. There is even an established "Linear Project Management Life Cycle" whose defining characteristic is that one process group finishes before the next begins—and it is explicitly described as *not* being a learning model.  PrepCast +2

That is almost the **opposite** of what you're describing.

Your idea is linear in its **representation of time and priority**, but iterative in its **method of learning**:

NOT THIS:

Requirements → Design → Build → Test → Release
WATERFALL

BUT THIS:

—————→ TIME
Explore → Try → Learn → Adjust → Build → Observe → Learn → Adjust ..
[—— current window ——]

So I'd preserve **line/forward/sequence** as the metaphor while avoiding a name that sounds like waterfall.

What seems distinctive about the method

I think there are actually four ideas here, and the eventual name should probably emphasize one or two of them.

- 1. One ordered line of work.** There is a canonical sequence rather than a collection of buckets. Priority is expressed spatially: *this comes before that*.
- 2. Iteration unrolled through time.** PDSA isn't represented as a perpetual circle detached from history. Each pass through the cycle produces new work and knowledge farther along the line.
- 3. The moving window.** This might actually be the most original/interesting metaphor. The project can be huge, but our useful field of attention is local. The window moves naturally forward as work progresses.
- 4. Past and future are the same structure.** This is particularly strong. A project doesn't switch from "plan" into "archive." The line is both. To understand March, look at March. To understand what's next, look ahead. There is no infinitely growing Done bucket.

And there's a philosophical idea underneath all four:

Time is the primary structure; status is metadata.

That feels much more distinctive than merely "tasks arranged in a line."

Names I'd explore

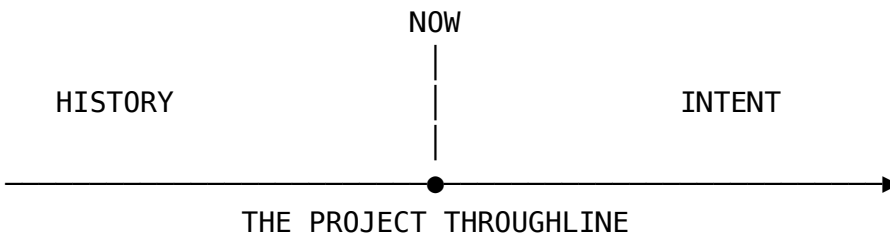
My strongest candidates right now are:

Name	Why I like it
Forward	Extremely simple. Captures progression and the arrow logo beautifully without implying waterfall.
Throughline	A continuous thread from history through now into the future. Really captures the persistence of the project record.
Project Line	Very literal, easy to explain, and could become "the Line" conversationally.
Flowline	Combines continuous flow with the linear representation. Strong Lean/Kanban resonance without being Kanban.
Timeline	Conceptually almost perfect, but far too generic/product-conflicted.
Continuum	Past/present/future as one continuous structure. A little abstract, but philosophically right.
Progression	Emphasizes movement without implying fixed phases.
Sequence	Very clean statement of the fundamental data model. Possibly too generic.
Thread	One continuous history of what happened and what comes next. Nice software metaphor too.
Unroll	Captures the PDSA insight specifically: iterative work unrolled onto time. Quirky and memorable.
Forward Flow	More descriptive and nicely distances the idea from waterfall.

Name	Why I like it
Continuous Planning	Accurately captures the rolling planning aspect, though less distinctive as a brand.

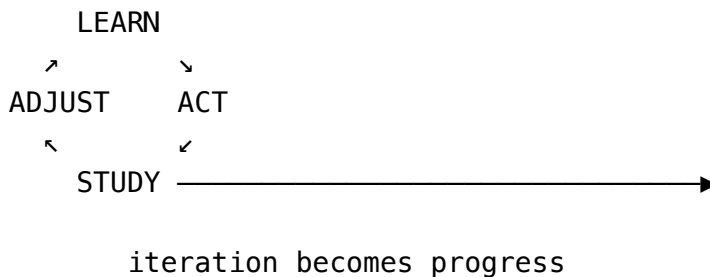
I particularly like **Throughline**, **Forward**, and **Flowline**, but for different purposes.

Throughline might be the best *methodology* name. A throughline is something that connects the beginning, middle, and end into a coherent whole. That is remarkably close to the model:



Then you can develop terminology naturally around it: **the Line, Now, the Window, move forward, look back, look ahead, zoom out.**

Forward is probably the strongest *product* name of those. And your logo already communicates it: a cycle literally resolves into a forward arrow. That's an unusually good visual encapsulation of the philosophy:



Flowline may be the best compromise. "Flow" connects it intellectually to Lean/Kanban and continuous work; "line" introduces the fundamental difference. It suggests **continuous, adaptive flow expressed as an ordered line**, rather than waterfall's predetermined sequence of phases.

There's also an interesting naming split

We don't necessarily need the **product**, **method**, and **central visualization** to have the same name.

In fact, I'd be inclined *not* to.

For example:

Product: Forward

Method: Throughline Project Management / Throughline Method

Core abstraction: The Line

Current working area: The Window

That gives you a surprisingly coherent vocabulary:

"Everything goes on **the Line**."

"Move it forward on the Line."

"That's outside the current **Window**."

"Look back on the Line and see what actually happened."

"Don't create a post-launch bucket. Put it after launch."

"Don't put it in Blocked. Decide where it belongs on the Line."

"The plan gets less certain as you look farther forward."

And perhaps the most important phrase:

Cycles of learning. A line of progress.

That also explains the logo beautifully. The curved portion isn't merely a funky arrow—it represents the iterative learning cycle. The straight arrow represents accumulated forward progress.

So I'd temporarily stop calling the concept **Linear Project Management** and use **Throughline** as the working method name while we develop it. It avoids the waterfall collision and, more importantly, describes what is actually novel here: **an adaptive, iterative project represented as one continuous line through past, present, and future.**

  Sources

ChatGPT can make mistakes. Check important info.